QuickJS

Bring JavaScript expressions into DuckDB queries

Install & load

```
SET allow_extension_repositories = 'allowed';
CREATE EXTENSION REPOSITORY IF NOT EXISTS query_farm WITH PREFIX 'https://duckdb-extensions.query.farm';
INSTALL quickjs FROM query_farm;  LOAD quickjs FROM query_farm;
```

01 At a glance

Evaluate a JavaScript expression, call a function with SQL arguments, or expand an array into rows. Use the embedded QuickJS engine for transformations that benefit from JavaScript control flow, string methods, or JSON reshaping.

SQL arguments → JavaScript → Text / JSON / rows

02 Core SQL surface

`SELECT quickjs(code)`

Evaluate JavaScript and return its result as VARCHAR.

`FROM quickjs(code, ...)`

Evaluate an array expression; emit one result row per element.

`quickjs_eval(function, ...)`

Call a JavaScript function with native arguments; return JSON, or NULL for undefined.

`arg0, arg1, ...`

Access positional arguments in the table form.

`parsed_arg0, ...`

Access a parsed copy of JSON strings in the table form.

`SET quickjs_timeout_ms / quickjs_memory_limit`

Bound each evaluation (no timeout and 256MB by default); Ctrl-C also cancels.

03 Keep in mind

Match the return shape

Scalar quickjs returns VARCHAR, quickjs_eval returns JSON, and FROM quickjs returns rows in a result column. Arguments arrive as numbers, BigInt, Date, arrays and objects; JSON text stays a string. Cast or extract results explicitly.

04 Call a function with row values

```
SELECT n, quickjs_eval(
  'x ⇒ x * x', n
)::INTEGER AS square
FROM range(1, 4) t(n);
-- 1, 4, 9
```

Use function parameters for per-row SQL arguments.

05 Unpack a transformed array

```
SELECT result::INTEGER AS value
FROM quickjs(
  'parsed_arg0.map(x ⇒ x * arg1)',
  '[1,2,3]', 3
);
-- 3, 6, 9
```

The table form exposes raw and parsed arguments.

06 Reshape a struct

```
SELECT quickjs_eval(
  'd ⇒ ({user: d.name, tags: d.tags.length})',
  {'name': 'Ada', 'tags': ['x', 'y']}
);
-- {"user":"Ada","tags":2}
```

Structs and lists arrive as objects and arrays; JSON text still needs JSON.parse.

07 In practice

Keep the interpreter focused

JavaScript runs per row and costs more than native SQL. Filter first and reserve it for transformations that need it. This is an embedded language runtime, without browser or Node APIs; use trusted script text.

Full docs & examples ↗ Source ↗